

Enterprise Configuration & Parameter Management

for Agentic AI Platforms on AWS

2026 Edition

A comprehensive architecture-level research report covering how modern enterprises design, govern, secure, and operate centralized configuration management for large-scale Agentic AI platforms.

Document Type	Architecture Reference
Scope	Enterprise Agent Builder Platform
Primary Cloud	AWS (multi-cloud considerations included)
Classification	Internal — Architecture Review
Edition	2026 v1.0

Table of Contents

Executive Summary	Part 0
Part 1 — Configuration Taxonomy	Part 1
■ Infrastructure & Application Configuration ■ Runtime, Agent & Workflow Configuration ■ Security, Compliance & Governance Configuration ■ AI-Specific Configuration Categories	
Part 2 — Enterprise Architecture Patterns	Part 2
■ Configuration as Code & GitOps ■ Dynamic Runtime & Externalized Configuration ■ Progressive Delivery & Feature Flags ■ Internal Developer Platforms & Golden Paths	
Part 3 — AWS Services Deep Dive	Part 3
■ AWS Systems Manager Parameter Store ■ AWS AppConfig ■ AWS Secrets Manager ■ AWS Cloud Map & Service Discovery ■ DynamoDB, S3 & EventBridge for Configuration ■ Bedrock AgentCore Runtime ■ ECS / EKS / Lambda Integration	
Part 4 — Runtime Configuration	Part 4
■ Dynamic Refresh Patterns ■ Push vs Pull Models ■ Configuration Cache & Distributed Cache ■ Failure Recovery & Rollback	
Part 5 — Feature Flag Platforms	Part 5
■ AWS AppConfig Feature Flags ■ LaunchDarkly, Split, Unleash, OpenFeature ■ Progressive Rollout Strategies	
Part 6 — Secrets Management	Part 6
■ AWS Secrets Manager Deep Dive ■ HashiCorp Vault ■ Cross-Account & Multi-Region Secrets ■ Dynamic Secrets & Certificate Management	
Part 7 — Configuration Hierarchy Design	Part 7
■ Hierarchy Levels & Inheritance ■ Override Precedence & Conflict Resolution ■ Version Management & Dependencies	

Part 8 — Configuration Schema Design[Part 8](#)

- Agent, Workflow & Prompt Schemas
- Model, Policy & Knowledge Base Schemas
- Schema Evolution & Backward Compatibility

Part 9 — Configuration Lifecycle[Part 9](#)

- Design through Deployment
- Versioning, Rollback & Archival
- Governance, Ownership & Audit

Part 10 — Platform Engineering[Part 10](#)

- Internal Developer Platform Design
- Self-Service Portal & Configuration APIs
- GitOps, Terraform & Service Catalog

Part 11 — Security & Zero Trust[Part 11](#)

- ABAC, RBAC & Cedar Policy
- Config Encryption & Tamper Detection
- Supply Chain Security & Provenance

Part 12 — Observability[Part 12](#)

- Configuration Change Monitoring
- Propagation Delay & Failure Detection
- Audit Logs & Compliance Reports

Part 13 — AI-Specific Requirements[Part 13](#)

- Prompt Registry & Versioning
- Model Registry & Routing
- MCP, Tool & Agent Registries
- RAI Policies & Safety Rules

Part 14 — Configuration Delivery[Part 14](#)

- Pull, Push & Streaming Models
- EventBridge, Kafka & Redis Patterns
- Hybrid Delivery Best Practices

Part 15 — Developer Experience[Part 15](#)

- SDK Design & REST/GraphQL APIs
- Hot Reload & Offline Mode
- Testing & Mocking Configuration

Part 16 — Anti-Patterns[Part 16](#)

- Configuration Sprawl & Secret Leakage
- Common Enterprise Failures

Part 17 — Best Practices Catalog[Part 17](#)

- Naming Standards & Hierarchy
- Ownership, Documentation & Governance
- Security, Resilience & Automation

Part 18 — Comparison Matrix (40+ Criteria)	Part 18
Part 19 — Reference Architecture	Part 19
■ Configuration Control Plane Design ■ Multi-Account AWS Organizations ■ Sequence Diagrams ■ High Availability & Disaster Recovery	
Decision Matrix & Technology Selection	Appendix A
Implementation Roadmap (4 Phases)	Appendix B
RFC Template & Implementation Checklist	Appendix C

Executive Summary

As enterprises scale Agentic AI platforms to support hundreds or thousands of autonomous agents across multiple teams, environments, and cloud regions, configuration management emerges as a foundational architectural discipline — one that determines whether those platforms are safe, governable, cost-effective, and operationally resilient.

This report provides a comprehensive, architecture-level analysis of how leading organizations design, govern, secure, and operate centralized configuration management for large-scale Agentic AI platforms on AWS. It synthesizes lessons from AWS, Microsoft, Google, Netflix, Uber, Airbnb, LinkedIn, Anthropic, Databricks, and others where publicly available, producing actionable recommendations for enterprise Agent Builder platforms.

Key Findings

- **Configuration is a first-class platform concern.** Modern Agentic AI systems require more configuration categories (20+) than traditional applications, spanning infrastructure, runtime, model, prompt, memory, RAI policy, cost governance, and compliance domains. Treating configuration as an afterthought leads to sprawl, drift, and safety incidents.
- **No single AWS service solves the complete problem.** The optimal architecture combines AWS AppConfig (dynamic flags and feature rollout), Parameter Store (static non-sensitive values), Secrets Manager (credentials and API keys), DynamoDB (custom configuration service), and S3 (large configuration artifacts) in a layered, purpose-built configuration plane.
- **Runtime hot-reloading is non-negotiable for AI agents.** Unlike traditional microservices, AI agents must update LLM model selections, prompt versions, RAI guardrails, knowledge base references, and kill switches without redeployment. This requires event-driven refresh mechanisms anchored to EventBridge, SQS, and local configuration caches.
- **Hierarchical configuration with inheritance is the enterprise standard.** A 14-level hierarchy (Enterprise → Org → BU → Platform → Environment → Region → Tenant → Project → Agent → Workflow → Tool → Session → User → Request) enables reuse, override, and governance at every scope without configuration duplication.
- **Security must be embedded by design.** Configuration planes must implement Zero Trust with ABAC using AWS IAM and Cedar policy, short-lived credentials, configuration signing (KMS), tamper detection, and complete audit trails via CloudTrail and OpenTelemetry.
- **Developer experience determines adoption.** Platform teams must invest in self-service portals, typed SDKs, CLI tooling, GitOps workflows, and configuration testing frameworks. Poor DX causes teams to bypass governance and hardcode values — the most dangerous anti-pattern for AI systems.
- **Progressive delivery is mandatory for AI configuration changes.** Prompt version updates, model changes, and guardrail modifications must support canary rollout, A/B testing, and instant kill switches. A bad prompt update can cause cascading failures across all agent instances globally within seconds.

Strategic Recommendations

Phase	Label	Key Actions
Immediate (0–90 days)	PoC	Establish naming conventions, Parameter Store hierarchy, and Secrets Manager baseline. Implement AppConfig for feature flags. Create configuration ownership model.
Short-term (90–180 days)	MVP	Deploy centralized Configuration Control Plane on ECS/EKS. Implement GitOps via CodePipeline. Build developer SDK. Enable AppConfig agents with local caching.
Medium-term (180–365 days)	Enterprise Scale	Implement full 14-level hierarchy with inheritance engine. Deploy Prompt Registry with versioning and approval workflows. Enable canary rollout for all configuration types.
Long-term (12–24 months)	Global	Achieve global multi-region configuration federation. Implement AI-driven configuration optimization. Deploy configuration marketplace for cross-team reuse. Enable sovereign deployments with local configuration replicas.

■ Critical Risk: Enterprise Agent Builder Platforms without proper configuration governance face significant risks including prompt injection via misconfigured guardrails, cost overruns from uncontrolled model selection, compliance violations from misconfigured RAI policies, and cascading failures from uncontrolled configuration updates. The investment in a proper configuration plane pays back within the first quarter of operation at scale.

PART 1

Configuration Taxonomy

What Exactly Constitutes Configuration in Modern Agentic AI Systems

Modern Agentic AI systems require a dramatically expanded understanding of configuration. Unlike traditional applications where configuration means environment variables and database connection strings, AI agent platforms must manage 22 distinct configuration categories across the full agent lifecycle — from infrastructure provisioning to real-time safety policy enforcement.

1. Infrastructure Configuration

- | | |
|--|--|
| <ul style="list-style-type: none">• VPC IDs, subnet IDs, security group IDs• ECS cluster ARNs, EKS cluster endpoints, Lambda function ARNs• Load balancer ARNs, target group ARNs• RDS endpoints, ElastiCache cluster endpoints | <ul style="list-style-type: none">• S3 bucket names, DynamoDB table names• Regional endpoint overrides• Availability zone configuration• Auto-scaling policies and thresholds |
|--|--|

2. Application Configuration

- | | |
|--|---|
| <ul style="list-style-type: none">• Service endpoints and base URLs• Thread pool sizes, connection pool sizes• Cache TTLs and eviction policies• Circuit breaker thresholds | <ul style="list-style-type: none">• Retry policies (backoff strategy, max retries, jitter)• Timeout configurations (connection, read, write)• Batch sizes and parallelism settings• Health check intervals |
|--|---|

3. Runtime Configuration

- | | |
|--|--|
| <ul style="list-style-type: none">• Feature flags (agent capabilities on/off)• Kill switches (emergency stop for agents or tools)• A/B test assignments• Canary rollout percentages | <ul style="list-style-type: none">• Rate limits per tenant/agent/user• Traffic routing weights• Experiment group assignments• Dark launch flags |
|--|--|

4. Agent Configuration

- | | |
|---|--|
| <ul style="list-style-type: none">• Agent identity and description• Agent role and permission group• Max concurrent sessions• Agent behavioral constraints | <ul style="list-style-type: none">• Human approval thresholds (when to escalate)• Memory type (short-term / long-term / episodic)• Planning strategy (ReAct / Plan-and-Execute / Tree-of-Thought)• Multi-agent routing rules and orchestration mode |
|---|--|

5. Workflow Configuration

- Step definitions and DAG structure
- Conditional branching rules
- Parallel execution limits
- Step timeout and retry policies
- Compensation/rollback actions
- Human-in-the-loop trigger conditions
- Workflow version and activation flags
- Input/output schema references

6. Tool Configuration

- Tool endpoint URLs (REST/gRPC/MCP)
- Tool authentication method and credential references
- Tool input/output schemas
- Tool rate limits and quota settings
- Tool health check endpoints
- Tool timeout and retry policy
- Tool capability declarations
- Tool version pinning

7. Prompt Configuration

- Prompt template versions and ARNs
- System prompt references (Prompt Registry)
- Few-shot example references
- Output format specifications
- Chain-of-thought instructions
- Persona definitions
- Language and localization settings
- Prompt approval status and rollback version

8. Model Configuration

- Foundation Model IDs (Bedrock model IDs)
- Model version pinning
- Temperature, top-p, top-k sampling parameters
- Max token limits (input + output)
- Stop sequences
- Response format (JSON mode, structured output)
- Model fallback chains
- Embedding model selection

9. Memory Configuration

- Short-term memory window size (context tokens)
- Long-term memory store references (vector DB ARNs)
- Memory retrieval strategy (similarity threshold, top-k)
- Memory TTL and expiration policies
- Memory compression strategy
- Session persistence settings
- Cross-session memory policies
- Memory access control rules

10. Security Configuration

- IAM role ARNs and permission boundaries
- OAuth 2.0 client IDs and token endpoint URLs
- API Gateway authorizer references
- VPC endpoint configurations
- TLS certificate ARNs
- KMS key ARNs for encryption
- WAF rule group references
- Network firewall policy ARNs

11. Authorization Configuration

- RBAC role definitions
- ABAC attribute schemas
- Cedar policy store references
- OPA policy bundle URLs
- Permission group membership rules
- Scope definitions for OAuth
- Resource-level permission matrices
- Cross-tenant isolation rules

12. Feature Configuration

- Feature flags per agent type
- Capability enablement by environment
- Beta feature opt-in lists
- Graduated rollout targeting rules
- Dependency flags (features requiring other features)
- Feature expiration dates
- Feature flag evaluation context schema
- Override rules for specific tenants/users

13. Experiment Configuration

- Experiment IDs and hypothesis definitions
- Treatment assignment rules
- Sample size and statistical power settings
- Metric collection configuration
- Experiment start/end dates
- Early stopping criteria
- Holdout group configuration
- Result analysis endpoints

14. Cost Governance Configuration

- Monthly token budget per agent/tenant
- Cost alert thresholds
- Model tier restrictions (e.g., no GPT-4 in dev)
- Batch inference vs real-time routing thresholds
- Token counting strategy
- Cost allocation tags
- Budget exhaustion behavior (throttle vs block)
- Reserved capacity allocations

15. Observability Configuration

- OpenTelemetry collector endpoints
- Phoenix/Langfuse server URLs
- Trace sampling rates
- Log levels per component
- Metric export intervals
- Alert routing configurations
- Dashboard template references
- SLO definitions and error budgets

16. Compliance Configuration

- Data residency requirements (allowed regions)
- Data classification policies
- Retention periods per data type
- PII detection and masking rules
- Audit log requirements
- Regulatory framework references (SOC2, GDPR, HIPAA)
- Right-to-erasure policy references
- Consent management configurations

17. Operational Configuration

- On-call rotation references
- Incident severity thresholds
- Runbook URLs per failure mode
- Maintenance window definitions
- Deployment freeze periods
- Capacity reservation settings
- DR failover trigger conditions
- Chaos engineering parameters

18. Tenant Configuration

- Tenant ID and display name
- Tenant tier (Basic/Pro/Enterprise)
- Custom domain configurations
- Tenant-specific feature overrides
- Tenant data isolation mode (shared/dedicated)
- Billing account references
- Tenant-specific model allowlists
- White-label branding configuration

19. Environment Configuration

- Environment name (dev/test/uat/prod)
- Environment-specific endpoint overrides
- Debug flags (verbose logging in dev only)
- Mock service endpoints for testing
- Environment-specific secret store references
- Approval workflow requirements by environment
- Environment promotion rules
- Blue/green environment routing weights

20. Disaster Recovery Configuration

- RTO/RPO targets per service
- Backup schedule definitions
- Failover region priority order
- Data replication lag thresholds
- Warm standby vs cold standby settings
- Configuration snapshot frequency
- DR test schedule and runbook references
- Cross-region replication ARNs

21. Policy Configuration

- RAI (Responsible AI) policy references
- Guardrail configuration (Bedrock Guardrails ARNs)
- Constitutional AI policy document references
- Content moderation threshold settings
- Bias detection policy versions
- Toxicity filter configurations
- Output validation rule references
- Escalation policy for policy violations

22. Context Engineering Configuration

- Context window budget allocation strategy
- RAG retrieval configuration (top-k, similarity threshold)
- Knowledge Base ARNs and index references
- Context compression settings
- Context priority rules (recency vs relevance)
- Conversation history truncation strategy
- Dynamic context injection rules
- Context validation schemas

PART 2

Enterprise Architecture Patterns

How Leading Organizations Solve Configuration at Scale

Configuration as Code (CaC)

Configuration as Code treats every configuration artifact — parameter definitions, feature flag schemas, secret references, agent blueprints — as version-controlled source code with the same review, testing, and deployment rigor as application code.

- All configuration stored in Git repositories alongside infrastructure code
- Pull requests required for any configuration change (no direct console edits in prod)
- Automated linting, schema validation, and policy checks in CI pipelines
- GitOps operators (Flux, ArgoCD) reconcile desired state to actual state
- Immutable configuration artifacts (versioned S3 objects, Parameter Store versions)
- Configuration changes trigger deployment pipelines, not manual steps

Enterprise Adoption: Netflix pioneered CaC for their distributed systems with Archaius. AWS CodePipeline with CloudFormation/Terraform represents the AWS-native implementation. For AI platforms, prompt templates and agent schemas must be included in the CaC scope.

GitOps for Configuration

GitOps extends CaC by making Git the single source of truth for both infrastructure and configuration, with automated reconciliation ensuring the deployed state always matches the Git state. This is the enterprise standard for Kubernetes-based platforms.

- Git repository per environment (or branch-per-environment strategy)
- Automated drift detection alerts when deployed state diverges from Git
- Reconciliation loop continuously syncs configuration from Git to runtime
- Rollback is a Git revert — simple, auditable, and reversible
- Signed commits (GPG) enforce configuration provenance
- Branch protection rules prevent unauthorized configuration changes

Enterprise Adoption: Kubernetes-based Agent Builder platforms should use FluxCD or ArgoCD for GitOps. AWS CodePipeline with SSM Parameter Store provides an AWS-native alternative. Anthropic and similar AI infrastructure teams use GitOps for model deployment pipelines.

Externalized Configuration

The Externalized Configuration pattern (from the 12-Factor App methodology) separates configuration from code entirely, injecting all environment-specific values at runtime from an external configuration service rather than bundling them in container images or deployment packages.

- Zero hardcoded values in application code or container images
- Configuration injected via environment variables, mounted files, or SDK calls
- Configuration service acts as the authoritative source (AppConfig, Parameter Store)
- Applications fail fast at startup if required configuration is missing
- Configuration schema validation enforced at injection time
- Supports multiple configuration sources with clear precedence order

Enterprise Adoption: AWS Lambda, ECS, and EKS all support externalized configuration natively via environment variables, SSM Parameter Store integration, and Secrets Manager. The AWS AppConfig Agent extension provides a local proxy for ECS/EKS workloads.

Hierarchical Configuration with Inheritance

A hierarchical configuration model organizes configuration in a tree structure where lower levels inherit from higher levels and can selectively override specific values. This dramatically reduces duplication and simplifies multi-environment, multi-tenant configuration management.

- Enterprise-level defaults apply everywhere unless overridden
- Environment-level overrides (dev gets verbose logging, prod gets minimal)
- Tenant-level overrides (Enterprise tier gets different model limits)
- Agent-level overrides for specific behavioral customizations
- Merge strategy defined per configuration key (last-wins, deep-merge, append)
- Circular reference detection in inheritance chains

Enterprise Adoption: AWS AppConfig supports hierarchical configuration profiles. Spring Cloud Config pioneered this pattern in the Java ecosystem. For Agentic AI platforms, the hierarchy must extend to the agent, workflow, and session levels.

Progressive Delivery

Progressive delivery applies continuous deployment techniques — canary releases, feature flags, ring deployments, A/B tests — to configuration changes, allowing teams to validate configuration changes with a subset of agents/tenants before full rollout.

- Configuration changes deployed to canary agents (1% → 10% → 50% → 100%)

- Automated rollback on error rate increase or latency degradation
- Feature flags enable runtime toggling without deployment
- A/B testing framework measures impact of configuration changes
- Ring deployment: dev → internal users → beta customers → general availability
- Kill switches provide instant global rollback capability

Enterprise Adoption: LaunchDarkly and AWS AppConfig provide native progressive delivery for configuration. For AI platforms, prompt version rollouts and model upgrades must use progressive delivery to prevent quality regressions from reaching all users simultaneously.

Configuration Federation

Configuration Federation allows multiple teams and systems to own and manage different portions of the configuration namespace, while a central configuration plane provides unified access, governance, and observability across all configuration sources.

- Each platform team owns their configuration namespace
- Central platform provides discovery, caching, and governance layer
- Configuration consumers get a unified API regardless of backend source
- Cross-team configuration dependencies are explicitly declared and versioned
- Federation layer enforces naming conventions and schema standards
- Circular dependency detection across team boundaries

Enterprise Adoption: Uber's configuration management at scale uses federation where team-owned stores are aggregated by a central configuration mesh. For an Enterprise Agent Builder Platform, each AI team manages their agent configs while the platform team governs the shared guardrails, models, and security policies.

Policy-Driven Configuration

Policy-Driven Configuration uses declarative policy engines (OPA, Cedar, AWS SCPs) to automatically enforce constraints on configuration values — preventing invalid, unsafe, or non-compliant configurations from being deployed regardless of who submits the change.

- OPA policies validate configuration changes in CI pipelines
- Cedar policies enforce real-time configuration access control
- AWS Service Control Policies prevent cross-account configuration leakage
- Automated remediation for configuration drift
- Policy-as-code versioned alongside configuration
- Compliance reporting generated from policy evaluation results

Enterprise Adoption: HashiCorp Sentinel and AWS Config Rules implement policy-driven configuration. For Agentic AI platforms, policies must enforce RAI guardrail minimums, cost limit maximums, and required observability configuration.

Intent-Driven Configuration

Intent-Driven Configuration allows developers to declare what they need (high throughput agent, cost-optimized agent, safety-critical agent) and lets the platform translate intents into concrete configuration values — abstracting implementation details from agent developers.

- Agent developers specify intent profiles (FAST, SAFE, CHEAP, BALANCED)
- Platform translates intent to concrete model, prompt, and resource configuration
- Intent profiles are platform-maintained and optimized continuously
- Developers can override specific values while keeping intent profile baseline
- Intent profiles are versioned and A/B tested for quality
- New model releases automatically update intent profile mappings

Enterprise Adoption: This pattern is emerging in enterprise AI platforms where developers should not need to know specific Bedrock model IDs or optimal temperature settings. The platform maintains curated profiles that encode best practices and safety defaults.

Internal Developer Platform (IDP) for Configuration

An IDP provides the developer-facing interface for configuration management: a self-service portal, CLI tools, Terraform providers, and configuration APIs that make it easy to correctly create, update, and consume configuration without deep knowledge of underlying infrastructure.

- Self-service configuration portal with form-based agent creation
- Golden path templates for common agent archetypes
- Configuration CLI with tab completion and validation
- Terraform provider for infrastructure-as-code configuration
- Configuration marketplace for sharing reusable components
- Automated documentation generation from configuration schemas

Enterprise Adoption: Platform engineering teams at companies like Spotify (Backstage), Airbnb, and Shopify have built IDPs that include configuration management as a core capability. For Agentic AI platforms, Backstage plugins for agent configuration represent the current state of the art.

PART 3

AWS Services Deep Dive

Comprehensive Evaluation of AWS Configuration Services

AWS Systems Manager Parameter Store

Hierarchical key-value store for non-secret configuration values, supporting plain text (Standard Tier) and encrypted (SecureString) parameters.

Ease of Use	★4/5
Scalability	★3/5
Hot Reload	★2/5
Security	★4/5
Cost	★5/5
Feature Flags	★1/5
Governance	★3/5

- Latency:** 5–15ms (Standard), 15–30ms (SecureString/KMS)
- Availability:** 99.9% SLA
- Throughput:** 40–1000 TPS depending on tier
- Value size:** 4KB (Standard) / 8KB (Advanced)
- Versions:** Up to 100 per parameter
- Cost:** Free (Standard) / \$0.05 per Advanced parameter/month
- Hot Reload:** Not native — requires polling or EventBridge
- Rollback:** Manual version retrieval (GetParameter with version label)

✓ STRENGTHS	✗ WEAKNESSES
-------------	--------------

• Native AWS integration — works seamlessly with IAM, Lambda, ECS, EKS, CloudFormation • Hierarchical namespacing with path-based organization (/platform/env/service/key) • CloudFormation and Terraform native support (SSM Parameter references) • Free tier for Standard parameters (up to 10,000 parameters) • GetParametersByPath API enables bulk retrieval by prefix • Event-driven updates via EventBridge when parameters change • CloudTrail audit logging for all parameter access and modifications • AWS Lambda Extensions support for local parameter caching • Parameter versioning (up to 100 versions per parameter)	• No native dynamic refresh/push notification to running applications • API throughput limits: 40 TPS (Standard) / 1000 TPS (Advanced) — can bottleneck at scale • No feature flag semantics — purely key-value, no targeting or rollout logic • Maximum value size: 4KB (Standard) / 8KB (Advanced) • Cross-region replication requires custom implementation • No built-in schema validation or type safety • SecureString requires KMS calls which add latency • Hierarchical path limit: 15 levels deep • No native configuration grouping or atomic multi-parameter updates
USE FOR	AVOID FOR
→ Static non-sensitive configuration (endpoints, ARNs, region names) → CloudFormation/Terraform variable injection → Environment-specific baseline configuration → Infrastructure parameter sharing across accounts (cross-account SSM) → Application startup configuration (not runtime-hot-reloaded)	✗ Secrets and credentials (use Secrets Manager instead) ✗ Feature flags with targeting rules (use AppConfig) ✗ High-frequency runtime configuration reads (>100 TPS per parameter) ✗ Large configuration documents (>8KB) ✗ Configuration requiring atomic updates across multiple parameters

AWS AppConfig

Managed configuration deployment service with built-in progressive rollout, automated validation, deployment strategies, and real-time configuration distribution to running applications. The primary choice for dynamic, hot-reloadable configuration.

Ease of Use	★4/5
Scalability	★4/5
Hot Reload	★5/5
Security	★4/5
Cost	★3/5
Feature Flags	★4/5
Governance	★4/5

Latency: <1ms (agent cache hit), 10–50ms (cache miss / API call)

Availability: 99.9% SLA

Max Doc Size: 1MB

Cache TTL: Configurable (default 90s for AppConfig Agent)

Rollback: Automatic on CW alarm or manual stop-deployment

Cost: \$0.0008 per deployment + \$0.0008 per 1000 client calls

Hot Reload: Yes — long-poll or agent-based

Feature Flags: Native (simple), advanced requires Evidently/LaunchDarkly

✓ STRENGTHS	✗ WEAKNESSES
-------------	--------------

• Built-in deployment strategies: Linear, Exponential, AllAtOnce with configurable bake times • Automated rollback on CloudWatch alarm triggers • Configuration validation via Lambda validators or JSON Schema • AppConfig Agent (Lambda Extension / sidecar) provides local caching with sub-millisecond reads • Supports multiple configuration types: Feature Flags, Freeform (JSON/YAML/text) • Native integration with Bedrock, Lambda, ECS, EKS • Configuration environments and applications provide logical grouping • Long-poll API enables efficient change detection without excessive API calls • Canary and blue/green deployment support • AWS IAM and resource-based policies for fine-grained access control	• Maximum configuration document size: 1MB — insufficient for very large agent schemas • No native multi-level hierarchy (must implement via conventions) • Targeting rules (per-user/per-tenant flags) require AWS Evidently or LaunchDarkly • Pricing adds up at scale: \$0.0008 per configuration deployment + client calls • Learning curve for deployment strategy configuration • No built-in A/B testing or experiment framework • Configuration retrieval requires knowledge of application/environment/profile structure • Limited querying capabilities (no configuration search)
USE FOR	AVOID FOR
→ Feature flags and kill switches for agents and tools → Dynamic runtime configuration (prompt versions, model parameters) → Progressive rollout of configuration changes (canary, linear) → Any configuration that must update without agent redeployment → Freeform JSON configuration documents for complex agent schemas	✗ Secrets and credentials (use Secrets Manager) ✗ Infrastructure-level configuration (CloudFormation parameters) ✗ Very large configuration documents (>1MB) ✗ Fine-grained user/tenant targeting without Evidently ✗ High-frequency writes (AppConfig is primarily read-optimized)

AWS Secrets Manager

Managed secrets storage with automatic rotation, cross-account access, multi-region replication, and tight IAM integration for managing credentials, API keys, OAuth client secrets, and other sensitive configuration values.

Ease of Use	★4/5
Scalability	★4/5
Hot Reload	★3/5
Security	★5/5
Cost	★2/5
Feature Flags	★1/5
Governance	★5/5

- Latency:** 20–50ms (API), 1–5ms (SDK cache hit)
- Availability:** 99.99% SLA
- Max Size:** 65KB per secret
- Rotation:** Built-in for RDS/Redshift, Lambda for custom
- Replication:** Multi-region (eventual consistency)
- Cost:** \$0.40/secret/month + \$0.05/10K API calls
- Hot Reload:** Via SDK caching client (configurable TTL)
- Audit:** Full CloudTrail logging

✓ STRENGTHS	✗ WEAKNESSES
• Automatic secret rotation for supported databases (RDS, Redshift, Elasticsearch) • Custom Lambda rotation functions for any secret type (OAuth tokens, API keys) • Cross-account access via resource-based policies without credential sharing • Multi-region secret replication for global applications • AWS SDK caching client reduces API calls and latency • Fine-grained IAM policies at secret level (including VPC conditions) • Complete CloudTrail audit log for all secret access • Secret versioning with staging labels (AWSCURRENT, AWSPENDING, AWSPREVIOUS) • Integration with Parameter Store (for referencing secrets from CloudFormation) • Tag-based access control enables ABAC patterns	• Cost: \$0.40 per secret/month + \$0.05 per 10,000 API calls — expensive at scale • No native dynamic injection (applications must call API or use SDK caching client) • Rotation requires careful implementation to avoid race conditions • No secret templating or secret schema validation • Cross-region replication is eventually consistent (not real-time) • Maximum secret size: 65KB • Not designed for non-secret configuration (use Parameter Store) • Rotation windows can cause brief downtime if not implemented correctly
USE FOR	AVOID FOR
→ Database passwords (RDS, Aurora, DynamoDB DAX) → OAuth client secrets and tokens → API keys for external services (OpenAI, Anthropic, third-party tools) → TLS certificate private keys → Encryption keys that need rotation → Service-to-service credentials	✗ Non-sensitive configuration values (use Parameter Store — 10x cheaper) ✗ High-frequency reads without caching (expensive at scale) ✗ Feature flags or dynamic runtime configuration ✗ Configuration requiring sub-millisecond access without caching

AWS AppConfig with Evidently

AWS CloudWatch Evidently provides enterprise feature flagging, A/B testing, and experimentation on top of AppConfig, enabling per-user/per-tenant targeting with statistical analysis of configuration change impacts.

Ease of Use	★3/5
Scalability	★4/5
Hot Reload	★4/5
Security	★4/5
Cost	★3/5
Feature Flags	★4/5
Governance	★3/5

- Targeting:** Segment-based (user attributes, custom rules)
- Rollout:** Percentage-based with traffic splits
- A/B Testing:** Native statistical significance
- Integration:** CloudWatch Metrics, AppConfig
- Cost:** Pay-per-evaluation after free tier
- Hot Reload:** Yes, via AppConfig

✓ STRENGTHS	✗ WEAKNESSES
-------------	--------------

• Percentage-based feature rollout with targeting rules • Built-in A/B testing with statistical significance calculation • Integration with AppConfig for configuration delivery • Real-time experiment metric collection via CloudWatch • Segment-based targeting (by user attributes, tenant tier, region) • Overrides for specific users/tenants (whitelist/blacklist) • Launch events for progressive feature enablement • No SDK dependency for basic flag evaluation (REST API)	• Less mature than LaunchDarkly for complex targeting scenarios • SDK ecosystem smaller than commercial alternatives • No built-in code reference management • Experiment analysis requires CloudWatch expertise • No native integration with third-party analytics platforms • Pricing model can be complex for large-scale experiments
USE FOR	AVOID FOR
→ A/B testing prompt variants → Progressive rollout of new model versions → Tenant-tier based feature enablement → Experiment-driven agent capability rollout	✗ Complex multi-dimensional targeting (use LaunchDarkly) ✗ Non-AWS environments ✗ Require rich SDK ecosystem across many languages

DynamoDB-backed Configuration Service

A custom configuration microservice built on DynamoDB provides maximum flexibility for enterprise-specific configuration schemas, complex queries, hierarchical inheritance, and integration with existing governance workflows.

Ease of Use	★2/5
Scalability	★5/5
Hot Reload	★5/5
Security	★4/5
Cost	★3/5
Feature Flags	★3/5
Governance	★4/5

- Latency:** <1ms (DAX cache), 1–3ms (DynamoDB direct)
- Availability:** 99.999% (Global Tables)
- Throughput:** Unlimited (with proper key design)
- Item Size:** 400KB per item
- Hot Reload:** Yes (via Streams + Lambda/SNS/SQS/EventBridge)
- Replication:** Global Tables (active-active, <1s replication)
- Cost:** Variable — DynamoDB pricing + service infrastructure
- Rollback:** Custom implementation required

✓ STRENGTHS	✗ WEAKNESSES
-------------	--------------

• Unlimited flexibility for configuration schema and query patterns • DynamoDB Streams enable real-time push notification of configuration changes • Single-digit millisecond latency with DAX caching • Hierarchical configuration via DynamoDB partition/sort key design • Native support for complex configuration documents (JSON up to 400KB per item) • Global Tables provide multi-region active-active replication • DynamoDB Streams + Lambda enable event-driven configuration propagation • No vendor lock-in to specific configuration semantics • Can implement custom inheritance, conflict resolution, and versioning logic • Point-in-time recovery for configuration history	• Requires building and maintaining a custom service (significant engineering investment) • Need to implement all governance, validation, and rollout logic from scratch • DynamoDB table design is critical — mistakes are costly to fix • Must implement own SDK, API, and developer tooling • Operational overhead of managing the service itself • Requires careful capacity planning (on-demand vs provisioned throughput) • Hot partition risk if many agents read the same configuration key simultaneously
USE FOR	AVOID FOR
→ Complex configuration schemas not supported by AppConfig (>1MB, complex hierarchies) → Configuration requiring complex query patterns (search by agent type, tenant, capability) → Multi-tenant configuration with complex inheritance rules → Configuration marketplace — searchable, tagged, reusable components → Audit history and configuration lineage tracking	✗ Simple key-value configuration (use Parameter Store — lower operational overhead) ✗ Secrets (use Secrets Manager) ✗ Teams without DynamoDB expertise ✗ MVP/PoC phase — too much engineering investment up-front

Bedrock AgentCore Runtime

AWS Bedrock AgentCore Runtime provides managed infrastructure for deploying AI agents at scale, with built-in configuration integration, memory management, tool connectivity, and observability — specifically designed for Agentic AI workloads.

Ease of Use	★5/5
Scalability	★4/5
Hot Reload	★4/5
Security	★5/5
Cost	★3/5
Feature Flags	★3/5
Governance	★4/5

Launch Year: 2024 (GA 2025)

Scaling: Auto-scaling (managed)

Model Support: Bedrock-native models

Tool Integration: Lambda, API, MCP

Observability: CloudWatch, X-Ray native

Memory: Built-in session + external store

✓ STRENGTHS	✗ WEAKNESSES
-------------	--------------

• Native integration with Bedrock models, knowledge bases, and guardrails • Built-in session management and agent memory • Managed tool connectivity (API, Lambda, MCP server integration) • Auto-scaling for concurrent agent execution • Native observability integration with CloudWatch and X-Ray • IAM-native access control for agent permissions • Integration with Parameter Store and Secrets Manager for agent configuration • Multi-agent orchestration support • Built-in retry and timeout handling • Bedrock Guardrails integration for real-time safety enforcement	• Relatively new service (2024/2025) — not yet production-proven at large enterprise scale • Limited to AWS Bedrock model ecosystem (not model-agnostic) • Less flexibility for custom agent frameworks (LangChain, custom Python) • Vendor lock-in risk for core agent orchestration • Pricing model still evolving • Limited support for complex multi-step workflow orchestration
USE FOR	AVOID FOR
→ AWS-native agent deployments using Bedrock models → Rapid agent deployment without custom infrastructure → Agents requiring built-in memory and tool management → Enterprises wanting fully managed agent runtime	✗ Multi-cloud or model-agnostic agent frameworks ✗ Complex custom agent orchestration patterns ✗ Organizations with strict vendor lock-in policies

PART 4

Runtime Configuration

How Enterprises Update Configuration Without Redeployment

The ability to update configuration at runtime — without redeploying application code — is one of the most critical capabilities for enterprise Agentic AI platforms. A misconfigured prompt, a model that produces poor outputs, or an enabled capability that causes compliance issues must be correctable in seconds, not hours.

Configuration Refresh Patterns

Pull-Based Polling

Applications periodically poll the configuration service for updates. Simple to implement but introduces update latency equal to the polling interval.

- Polling interval: 30s–300s depending on criticality
- Exponential backoff on configuration service failures
- Last-known-good cache prevents failures if service is unavailable
- Jitter added to polling interval to prevent thundering herd
- Checksum/ETag comparison avoids unnecessary deserialization
- AppConfig Agent uses efficient long-polling (not busy-polling)

→ Best for: *AppConfig (built-in long-poll), Parameter Store polling, low-criticality flags*

Push-Based Event-Driven

Configuration service pushes changes to subscribers via messaging systems. Near-real-time updates with minimal polling overhead, but requires message broker infrastructure.

- EventBridge rules trigger on Parameter Store / AppConfig changes
- SNS fan-out to per-service SQS queues for guaranteed delivery
- Lambda functions process configuration change events
- ECS/EKS sidecar containers receive updates and apply to agent config
- WebSocket connections for truly real-time configuration streaming
- Dead letter queues for failed configuration update processing

→ Best for: *kill switches (must propagate in <5s), emergency guardrail updates*

Local Configuration Cache

Each agent instance maintains an in-memory or on-disk cache of its configuration. This provides sub-millisecond configuration reads and resilience against configuration service outages.

- AppConfig Agent extension maintains local cache (default 90s TTL)
- Redis/ElastiCache as distributed cache for EKS agent fleets
- SQLite-backed local cache for Lambda (persistent across warm starts)
- Cache invalidation on configuration change events
- Stale-while-revalidate pattern: serve cached value while refreshing
- Cache warming on agent startup (pre-load all required configuration)

→ Critical: agents must NEVER fail if configuration service is temporarily unavailable

Configuration Snapshots

Point-in-time snapshots of complete configuration state, stored in S3. Agents boot from snapshots and apply incremental updates from the change stream.

- Daily full snapshots to S3 for disaster recovery
- Incremental change log for replay from any point in time
- Snapshot integrity verification (SHA-256 hash)
- Agents download and validate snapshot on cold start
- Snapshot compression for large configuration sets
- Cross-region snapshot replication for DR scenarios

→ Enables fast recovery and rollback to any previous configuration state

Distributed Cache Pattern

ElastiCache (Redis) or DAX (for DynamoDB-backed config) provides a shared distributed cache for agent fleets running on ECS/EKS, enabling consistent configuration reads across hundreds of instances.

- Redis Pub/Sub for real-time configuration change notification
- DAX cluster in front of DynamoDB for consistent <1ms reads
- Cache cluster sized for peak agent concurrency
- Redis Cluster mode for horizontal scaling
- Separate cache namespaces per tenant for isolation
- Cache warmup scripts for Blue/Green deployment switches

→ Required for large EKS deployments (100+ agent pods) to avoid Parameter Store TPS limits

Configuration Update Latency Targets

Different configuration types have different acceptable update latency windows. Enterprises must design their configuration delivery pipeline to meet these targets.

Configuration Type	Max Acceptable Latency	Recommended Mechanism	Failure Mode
Kill Switches	<5 seconds	EventBridge + Push	Block all requests

Configuration Type	Max Acceptable Latency	Recommended Mechanism	Failure Mode
Guardrail Updates	<30 seconds	AppConfig + Push	Use previous guardrail
Prompt Version Updates	<60 seconds	AppConfig polling	Use previous prompt version
Model Selection Changes	<60 seconds	AppConfig polling	Use default model
Feature Flags	<90 seconds	AppConfig Agent cache	Use default (off)
Cost Limits	<5 minutes	AppConfig polling	Use conservative default
Tool Endpoints	<15 minutes	Parameter Store polling	Use cached endpoint
Static Configuration	Restart cycle	Parameter Store / S3	Fail to start

Failure Recovery & Rollback

Configuration changes are the #1 cause of production incidents in distributed systems. Every configuration update must include a tested rollback path that can be executed in under 60 seconds.

- **AppConfig automatic rollback:** Configure CloudWatch alarms (error rate, latency) that trigger automatic deployment rollback via AppConfig stop-deployment API
- **Feature flag kill switch:** All feature flags have an emergency off switch that bypasses all targeting rules and immediately disables the feature globally
- **Configuration version labels:** Parameter Store versions labeled STABLE, CURRENT, CANARY — instant rollback by changing the CURRENT label to point to STABLE version
- **Snapshot rollback:** Agents can be instructed to boot from a specific configuration snapshot for full environment rollback
- **Git revert:** GitOps-managed configuration is rolled back via Git revert PR, automatically triggering the deployment pipeline
- **Blue/Green configuration swap:** Maintain two complete configuration sets (blue and green), instant switchover by changing routing configuration

PART 5

Feature Flag Platforms

Enterprise Feature Flag Architecture for Agentic AI

Feature flags are the control plane for progressive delivery of AI agent capabilities. Unlike traditional software where a bad feature causes a UI bug, a bad AI feature flag can expose unsafe agent behaviors to production users. Enterprise feature flag platforms for AI must support emergency kill switches, per-tenant targeting, and integration with RAI policy enforcement.

AWS AppConfig Feature Flags

Architecture	Managed AWS service storing feature flag definitions as JSON documents in AppConfig. Integrates natively with Lambda (extension), ECS/EKS (sidecar), and EC2. Evaluation happens client-side using downloaded flag document.
Targeting	Basic targeting via flag document structure. Advanced targeting requires CloudWatch Evidently (segment-based) or custom evaluation logic.
Rollout	Percentage-based via Evidently launches. Manual percentage changes via flag document updates with AppConfig deployment strategies.
Kill Switch	Update flag to false → AppConfig deployment → propagates in 90s (agent cache TTL). Emergency: set cache TTL to 0s for immediate propagation.
Pricing	\$0.0008 per configuration deployment + API call costs. Essentially free for low-volume flag operations.
Recommendation	USE for simple binary flags and AWS-only deployments. AUGMENT with Evidently for targeting. REPLACE with LaunchDarkly for complex enterprise needs.

LaunchDarkly

Architecture	SaaS feature management platform with edge SDK evaluation. Flag rules evaluated client-side from locally cached rule set streamed via SSE. Sub-millisecond flag evaluation latency. Enterprise adoption: Netflix, IBM, HP, Atlassian.
Targeting	Multi-dimensional targeting: user attributes, custom context types, percentage rollouts, targeting rules with boolean/string/number/JSON flag types. Tenant-based, region-based, agent-type-based targeting.

Rollout	Progressive rollout with percentage increments. Automated rollout based on metric thresholds. A/B testing with experiment management. Ring deployments.
Kill Switch	Instant kill: LD SDK maintains streaming connection. Flag off propagates in <500ms globally. Per-target kill switches supported.
Pricing	Enterprise pricing (custom). Developer tier from \$20/month. Seats-based pricing — can be expensive for large teams.
Recommendation	BEST choice for complex multi-tenant enterprise feature flags. Superior targeting, SDK ecosystem, and experiment management vs AWS-native options.

OpenFeature + Flagsmith

Architecture	OpenFeature is a CNCF standard API for feature flags that decouples application code from specific feature flag providers. Flagsmith is an open-source implementation deployable on AWS. Enables vendor switching without code changes.
Targeting	Flagsmith: rule-based targeting, percentage rollouts, identity-based flags. OpenFeature: provider-agnostic targeting via evaluation context.
Rollout	Gradual rollout in Flagsmith (percentage of users). A/B testing via segment configuration.
Kill Switch	Immediate flag disable in Flagsmith UI. Propagation depends on SDK polling interval (default 60s).
Pricing	Flagsmith: Open source (self-hosted free) or SaaS from \$45/month. OpenFeature: CNCF open source (free).
Recommendation	USE when vendor neutrality is required or budget is constrained. OpenFeature standard protects against future migration costs.

Unleash (Self-Hosted)

Architecture	Open-source feature flag platform (hosted or self-hosted on AWS). SDK available for Go, Python, Java, Node.js, .NET. PostgreSQL or MySQL backend. Enterprise version with SSO, RBAC, audit logs.
Targeting	Activation strategies: gradual rollout, user ID list, IP address, hostname, custom strategies. Variants for A/B testing.
Rollout	Gradual rollout strategy (0-100% by user ID hash). Variant-based rollout for A/B testing.
Kill Switch	Instant disable via API or UI. SDK polls every 15 seconds by default.
Pricing	Open source (free self-hosted). Enterprise from \$50/month for hosted.
Recommendation	USE for cost-sensitive deployments where LaunchDarkly pricing is prohibitive. Good for regulated industries requiring on-premise data residency.

AI-Specific Feature Flag Patterns

- **Model Version Flag:** Flag 'use_claude_4' enables routing to new model. Canary to 5% of agents → measure quality metrics → expand. Instant rollback if quality degrades.
- **Prompt Version Flag:** Flag 'prompt_version' returns 'v2.3.1' or 'v2.2.0'. AppConfig deployment strategy with 30-minute bake time and CW alarm rollback.
- **RAI Guardrail Flag:** Flag 'enable_strict_guardrails' overrides default. Emergency ON switch that bypasses all targeting rules — any compliance event enables globally.
- **Tool Availability Flag:** Flag 'enable_web_search_tool' controls tool availability per agent type and tenant tier. Gradual rollout with error rate monitoring.
- **Human Approval Flag:** Flag 'require_human_approval_for_financial' — kill switch that forces human review for all financial operations, regardless of configured threshold.
- **Cost Control Flag:** Flag 'enforce_token_budget' enables/disables strict token counting per agent. Emergency switch to prevent runaway cost events.

PART 6

Secrets Management

Enterprise Secrets Architecture for Agentic AI Platforms

Agentic AI platforms handle a large volume of secrets: LLM API keys, OAuth credentials for tool integrations, database passwords, vector database API keys, observability tokens, and inter-service authentication credentials. A mature secrets management architecture is non-negotiable for enterprise AI platforms.

Capability	AWS Secrets Manager	HashiCorp Vault	Azure Key Vault	Google Secret Manager
Auto Rotation	✓ Native (RDS/Lambda)	✓ Dynamic secrets	✓ Native (Azure resources)	Limited
Dynamic Secrets	✗ Static only	✓ Core feature	✗ Static only	✗ Static only
Cross-Cloud	AWS only	Multi-cloud	Azure focus	GCP focus
PKI / Certs	ACM integration	✓ Full PKI engine	✓ Certificate mgmt	✓ CA Service
Multi-Region	✓ Replication	✓ Enterprise	✓ Geo-redundancy	✓ Native
Audit Logging	✓ CloudTrail	✓ Audit log	✓ Activity log	✓ Cloud Audit
Short-Lived Creds	Limited (rotation)	✓ Core design	Limited	Limited
K8s Integration	External Secrets Op.	✓ Native (Agent)	✓ CSI driver	✓ Workload Identity
Cost (est)	\$0.40/secret/mo	Enterprise pricing	Azure pricing	\$0.06/version/mo
Complexity	Low	High	Medium	Medium
AWS IAM Integ	✓ Native	Via auth method	Limited	Limited

AWS Secrets Manager — Enterprise Implementation

Secret Naming Convention

Use hierarchical naming: `/environment/platform/service/secret-type`. Example: `/prod/agent-platform/bedrock/api-key`, `/prod/agent-platform/langfuse/token`, `/prod/agent-platform/oauth/{tool-name}/client-secret`. Tag every secret with: Environment, Platform, Team, DataClassification, RotationSchedule.

Automatic Rotation Architecture

Configure Lambda rotation functions for all API keys and OAuth tokens. Rotation Lambda: (1) Generate new credential, (2) Test new credential, (3) Update secret with AWSPENDING label, (4) Validate, (5) Promote to AWSCURRENT, (6) Delete old credential. Set rotation window: 30 days for API keys, 90 days for service accounts, 7 days for OAuth refresh tokens.

Cross-Account Access Pattern

Agent services in isolated AWS accounts access secrets via resource-based policies. Pattern: Secrets Manager resource policy allows specific IAM roles from child accounts. Avoids cross-account credential sharing. Audit log in the secrets owner account. Use AWS Organizations SCPs to enforce that secrets are only accessed from approved accounts.

SDK Caching Client

Use AWS Secrets Manager caching client in all agent code. Default cache TTL: 3600 seconds. Reduces API calls by 99%+ for frequently accessed secrets. Force refresh on rotation event via EventBridge → Lambda → cache invalidation. Never call GetSecretValue in hot paths — always use the caching client.

Dynamic Secrets via HashiCorp Vault

For advanced use cases, deploy HashiCorp Vault for dynamic secrets: Database secrets that auto-generate per-session credentials with 1-hour TTL. AWS STS credentials generated on-demand via Vault AWS secrets engine. Zero standing privileges — agents only get credentials when actively needed. Vault sidecar injection for EKS workloads via Agent Injector.

Secret Classification Matrix

Secret Type	Store	Rotation Period	Access Pattern	Risk Level
LLM API Keys	Secrets Manager	30 days	Read at startup (cached)	CRITICAL
OAuth Client Secrets	Secrets Manager	90 days	Read at startup	HIGH
Database Passwords	Secrets Manager	30 days (auto)	SDK cache	CRITICAL
Vector DB API Keys	Secrets Manager	30 days	Read at startup	HIGH
Service Account Keys	Secrets Manager / Vault	7 days	Dynamic (Vault)	HIGH
Observability Tokens	Secrets Manager	90 days	Read at startup	MEDIUM
MCP Server Auth Tokens	Secrets Manager	30 days	Read per session	HIGH
Encryption Keys	KMS (not SM)	Annual (KMS managed)	KMS API	CRITICAL
TLS Certificates	ACM / Vault	Annual (auto-renew)	ACM managed	CRITICAL
Webhook Signing Keys	Secrets Manager	30 days	Read at startup	HIGH

PART 7

Configuration Hierarchy Design

Organizing Configuration for Enterprise Scale

A well-designed configuration hierarchy is the cornerstone of scalable enterprise configuration management. It eliminates duplication, enables governed override, and provides clear ownership at every level of the organization.

The 14-Level Configuration Hierarchy

L1: Enterprise	Global defaults that apply to every agent across all organizations. Security minimums, compliance baselines, cost governance defaults, RAI policy minimums. Managed by: Platform Security & Architecture team. Change requires: Architecture Review Board approval.
L2: Organization	Organizational unit overrides. Business unit specific policies, regulatory constraints, approved model lists. Managed by: BU Platform Engineering. Change requires: BU CTO approval.
L3: Business Unit	BU-specific configuration: cost center codes, preferred observability platforms, BU-approved tool sets, internal API endpoints. Managed by: BU Platform team.
L4: Platform	Agent Builder Platform configuration: platform version, supported runtimes, platform feature flags, SDK versions. Managed by: Platform Engineering team.
L5: Environment	Environment-specific overrides: dev (verbose logging, mock endpoints), test (synthetic credentials), UAT (production models, test data), prod (real credentials, optimized settings). Managed by: DevOps/SRE team.
L6: Region	Regional configuration: Bedrock model availability by region, regional endpoints, data residency constraints, regional cost limits, latency-based routing. Managed by: Platform Engineering.
L7: Tenant	Tenant-specific configuration: tenant tier (Basic/Pro/Enterprise), tenant feature flags, custom model allowlists, tenant branding config, per-tenant token budgets. Managed by: Platform Operations.
L8: Project	Project-level configuration: project-specific tool sets, project knowledge bases, project prompt collections, project cost allocation. Managed by: Project team lead.

L9: Agent	Individual agent configuration: agent role, capabilities, model selection, prompt version, memory configuration, human approval thresholds. Managed by: Agent developer. Change requires: code review + AppConfig deployment.
L10: Workflow	Workflow-specific configuration: step definitions, branching rules, parallel limits, step timeouts. Managed by: Workflow developer.
L11: Tool	Tool-specific configuration: tool endpoints, authentication, rate limits, timeouts, retry policies. Managed by: Tool owner.
L12: Session	Session-level runtime configuration: session token budget, session memory scope, session feature overrides for A/B testing. Set programmatically at session creation.
L13: User	User-level preferences and overrides: language, persona, access tier, user-specific feature flags. Retrieved from user profile service.
L14: Request	Per-request runtime context: dynamic context injection, request-specific tool restrictions, real-time cost limits. Set by calling application at request time.

Parameter Store Hierarchy Implementation

The 14-level hierarchy maps directly to SSM Parameter Store paths:

```
/enterprise/defaults/security/min-guardrail-version
/enterprise/defaults/rai/content-policy-arn
/org/fintech/defaults/compliance/pii-detection-enabled
/platform/agent-builder/v2/supported-runtimes /env/prod/bedrock/model-allowlist
/region/us-east-1/bedrock/endpoint /tenant/{tenant-id}/token-budget/monthly-usd
/project/{project-id}/knowledge-base-arns /agent/{agent-id}/model-id
/agent/{agent-id}/prompt-version /agent/{agent-id}/memory-config
/workflow/{workflow-id}/step-timeout-seconds /tool/{tool-id}/endpoint-url
/tool/{tool-id}/auth-secret-arn
```

Inheritance Rules

- **Last-writer-wins:** Lower levels override higher levels for the same key
- **Deep merge for maps:** Nested JSON objects are merged recursively (not replaced)
- **Append for lists:** List values (e.g., allowed-tools) are unioned by default; explicit 'replace' prefix overrides
- **Explicit null to remove:** A lower level can remove an inherited value by explicitly setting null
- **Protected values:** Enterprise-level security minimums cannot be overridden (marked 'immutable' in schema)
- **Dependency resolution:** Configuration values referencing other values (e.g., `${env/prod/bedrock/endpoint}`) are resolved at evaluation time
- **Cycle detection:** Configuration resolver detects circular references and fails with descriptive error

PART 8

Configuration Schema Design

Type-Safe, Versioned Schemas for Every Configuration Category

Configuration schemas are the contract between configuration producers (platform teams, agent developers) and configuration consumers (agents at runtime). Strict JSON Schema validation with schema versioning prevents misconfiguration incidents and enables safe schema evolution over time.

Agent Configuration Schema

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "$id":
"https://platform.enterprise.com/schemas/agent/v1.2.0", "title": "AgentConfiguration",
"type": "object", "required": ["agentId", "modelConfig", "promptConfig", "memoryConfig",
"policyConfig"], "additionalProperties": false, "properties": { "agentId": { "type":
"string", "pattern": "^[a-z][a-z0-9-]{2,63}$" }, "version": { "type": "string",
"pattern": "^[\\d+\\.\\d+\\.\\d+\\d+$" }, "displayName": { "type": "string", "maxLength": 128
}, "description": { "type": "string", "maxLength": 1024 }, "modelConfig": { "type":
"object", "required": ["primaryModelId"], "properties": { "primaryModelId": { "type":
"string" }, "fallbackModelId": { "type": "string" }, "temperature": { "type": "number",
"minimum": 0, "maximum": 2 }, "maxTokens": { "type": "integer", "minimum": 1, "maximum":
200000 }, "topP": { "type": "number", "minimum": 0, "maximum": 1 }, "stopSequences": {
"type": "array", "items": { "type": "string" }, "maxItems": 10 } } }, "promptConfig": {
"type": "object", "required": ["systemPromptRef"], "properties": { "systemPromptRef": {
"$ref": "#/$defs/PromptRef" }, "fewShotRef": { "$ref": "#/$defs/PromptRef" },
"outputFormatRef": { "$ref": "#/$defs/PromptRef" } } }, "memoryConfig": { "type":
"object", "properties": { "shortTermWindowTokens": { "type": "integer", "minimum": 1000,
"maximum": 200000 }, "longTermStoreArn": { "type": "string", "pattern": "^arn:aws:.*" },
"retrievalTopK": { "type": "integer", "minimum": 1, "maximum": 20 },
"retrievalSimilarityThreshold": { "type": "number", "minimum": 0, "maximum": 1 } } },
"policyConfig": { "type": "object", "required": ["guardrailConfigArn", "raiPolicyRef"],
"properties": { "guardrailConfigArn": { "type": "string" }, "raiPolicyRef": { "type":
"string" }, "humanApprovalThreshold": { "type": "number", "minimum": 0, "maximum": 1 } }
}, "costConfig": { "type": "object", "properties": { "monthlyBudgetUSD": { "type":
"number", "minimum": 0 }, "budgetExhaustedBehavior": { "enum": ["THROTTLE", "BLOCK",
"NOTIFY_ONLY"] } } } }, "$defs": { "PromptRef": { "type": "object", "required":
["promptId", "version"], "properties": { "promptId": { "type": "string" }, "version": {
"type": "string" }, "registryArn": { "type": "string" } } } }
```

Schema Evolution Rules

- **MINOR version bump** (1.2.0 → 1.3.0): Adding optional fields with defaults. Fully backward compatible. Existing configurations remain valid.
- **MAJOR version bump** (1.x.x → 2.0.0): Removing required fields, renaming fields, changing field types. Requires migration script and dual-write period.
- **Schema registry**: All schema versions stored in S3 with immutable versioned paths. Agents specify the schema version they consume.
- **Validation pipeline**: Every configuration change validated against schema in CI. AppConfig Lambda validator enforces schema at deployment time.
- **Schema deprecation**: Deprecated schemas remain valid for 6 months with deprecation warning. Platform migrates consumers before end-of-life.
- **Breaking change process**: RFC required for breaking schema changes. Minimum 30-day notice. Migration tooling provided by platform team.

PART 9

Configuration Lifecycle

From Design Through Archival

Configuration has a complete lifecycle that must be managed with the same discipline as application code: design, review, approval, publishing, distribution, consumption, versioning, deprecation, and archival.

1. Design	Configuration schema drafted in YAML/JSON. Peer review by 2 platform engineers. Schema linting via CI pipeline. Owner assigned (team + individual). Documentation written.
2. Review & Approval	Pull request in GitOps repository. Automated checks: schema validation, naming convention, security policy, cost impact. Human review: platform team approval for cross-cutting config, owner review for agent-specific.
3. Publishing	Merge to main triggers CD pipeline. Parameter Store update via CloudFormation / Terraform apply. AppConfig deployment with configured deployment strategy (canary for prod). Event published to EventBridge (configuration.published event).
4. Distribution	AppConfig Agent delivers to running instances. EventBridge triggers Lambda to invalidate distributed caches. SNS notification to subscribed services. Configuration health check validates propagation.
5. Runtime Consumption	Agents read from local cache (AppConfig Agent). Cache miss triggers AppConfig API call. All reads logged to CloudWatch for observability. Schema validation on consumption (fail-fast on invalid config).
6. Versioning	Every change creates new version (immutable). Current version labeled ACTIVE. Previous version labeled PREVIOUS. Up to 100 versions retained in Parameter Store. Version history queryable via platform API.
7. Deprecation	Deprecation notice added to schema (deprecatedAt field). 6-month deprecation period with active migration support. Platform dashboard shows consumers still using deprecated configs. Automated PR creation for automated migrations.
8. Archival	Archived configurations moved to S3 Glacier after 90 days. Retained for 7 years for compliance. Audit log retained separately in CloudWatch Logs Insights. Archived configs can be restored for DR scenarios.

PART 10

Platform Engineering

Internal Developer Platform for Configuration

An Internal Developer Platform (IDP) for configuration management dramatically improves developer productivity and ensures configuration governance is embedded in the developer workflow rather than applied as a gate after the fact.

Self-Service Configuration Portal

- Web UI for browsing and creating agent configurations (Backstage plugin)
- Form-based agent builder with real-time schema validation
- Configuration diff viewer for change review
- One-click deployment to non-prod environments
- Approval workflow UI for production deployments
- Configuration health dashboard per agent/environment
- Usage analytics: which configurations are actively consumed

Configuration CLI (platform-config)

- `platform-config init agent --template bedrock-rag-agent`
- `platform-config validate --config agent.yaml --env prod`
- `platform-config deploy --config agent.yaml --env test --strategy canary`
- `platform-config rollback --agent my-agent --version v2.1.0 --env prod`
- `platform-config diff --agent my-agent --from prod --to test`
- `platform-config get --agent my-agent --env prod --key modelConfig.primaryModelId`
- `platform-config audit --agent my-agent --from 2025-01-01 --to 2025-12-31`

Terraform Provider (provider 'agentplatform')

- `resource 'agentplatform_agent' 'my_agent' { ... }` – full agent configuration as IaC
- `data 'agentplatform_model_list' 'approved' { tier = 'enterprise' }` – discover approved models
- `resource 'agentplatform_feature_flag' 'web_search' { ... }` – feature flags as code
- `resource 'agentplatform_prompt' 'system' { ... }` – prompt registration in IaC
- Automatic dependency resolution between related resources
- Import command for existing configurations

Configuration SDK (Python/TypeScript/Java/Go)

- `from agent_platform import ConfigClient, AgentConfig`
- `config = await ConfigClient.load(agent_id='my-agent', env='prod')`

```
→ model_id = config.model.primary_model_id # typed, IDE autocomplete  
→ config.on_change('modelConfig', lambda c: reload_model(c)) # hot reload  
→ config.validate() # raises ConfigValidationError on schema violation  
→ config.mock({'modelConfig.temperature': 0.0}) # for testing
```

Golden Path Templates

```
→ bedrock-rag-agent: RAG agent with Knowledge Base, Guardrails, memory  
→ bedrock-orchestrator: Multi-agent orchestration with routing rules  
→ tool-specialist: Single-purpose agent with specific tool set  
→ compliance-agent: Pre-configured with strict RAI and audit settings  
→ cost-optimized-agent: Haiku model, aggressive caching, token limits  
→ research-agent: High context window, web search, extended memory
```


PART 11

Security & Zero Trust

Securing Configuration in Agentic AI Platforms

Configuration security for Agentic AI platforms must address a unique threat model: adversarial prompt injection via misconfigured guardrails, cost attacks via model selection manipulation, data exfiltration via compromised tool endpoints, and unauthorized capability escalation via feature flag tampering.

Zero Trust Configuration Access

- No implicit trust — every configuration read requires explicit authorization
- IAM roles scoped to minimum required configuration paths
- VPC endpoints for all configuration service access (no public internet)
- mTLS for service-to-service configuration API calls
- Short-lived STS tokens for cross-account configuration access
- IAM conditions: `aws:SourceVpc`, `aws:RequestedRegion`, `aws:PrincipalTag`

ABAC with Cedar Policy

- Cedar policies define fine-grained access rules for configuration
- Example: permit agent with tag `AgentTier=='enterprise'` to read configuration in path `'/enterprise/**'`
- Configuration access context includes: agent-id, tenant-id, environment, IP range
- Policy evaluation cached in AVP (Amazon Verified Permissions)
- Policy as code — Cedar policies versioned in Git alongside configuration
- Real-time policy evaluation at configuration read time (not just write time)

Configuration Signing & Tamper Detection

- All configuration artifacts signed with KMS asymmetric key (RSA-2048)
- Agents verify signature before applying configuration
- SHA-256 hash stored alongside configuration for integrity verification
- Tamper detection: CloudWatch alarm on signature verification failures
- Configuration provenance: who created, who approved, who deployed
- SBOM linkage: configuration version linked to deploying pipeline run

Configuration Encryption

- All configuration at rest encrypted with customer-managed KMS keys
- KMS key per environment (separate keys for dev/test/uat/prod)
- KMS key policies enforce principle of least privilege
- Envelope encryption for large configuration documents (S3)
- SecureString parameters encrypted at Parameter Store level
- Configuration caches encrypted at rest (Redis AUTH + TLS, EBS encryption)

Configuration Drift Detection

- AWS Config Rules detect drift from approved configuration baselines
- Scheduled Lambda functions compare runtime config with Git source of truth
- Alerts on manual console edits that bypass GitOps pipeline
- Automated remediation: SNS alert + optional auto-rollback to Git state
- Configuration audit reports generated weekly for security review
- Drift score tracked as SLA metric for platform reliability

PART 12

Configuration Observability

Monitoring Configuration Changes, Propagation, and Failures

Configuration observability answers the critical operational questions: Did my configuration change propagate to all agents? Is the new prompt version being used? Did the kill switch activate globally? Why is Agent-X still using the old model?

Configuration Change Events

All configuration changes published to EventBridge as structured events. Schema: {eventType: 'configuration.updated', agentId, configPath, oldVersion, newVersion, deployedBy, deploymentStrategy, timestamp}. Events consumed by: audit log (CloudWatch), notification service (SNS), propagation tracker (DynamoDB), dashboard (Grafana).

Propagation Monitoring

After each configuration deployment, propagation tracker polls a random 10% sample of running agent instances to verify they have received the new version. Alert if <95% of instances have updated within SLA window (90s for AppConfig, 300s for polling). Dashboard shows propagation heatmap by region and availability zone.

Configuration Failure Detection

Agent SDK emits metric 'config.read.failure' on any configuration read error. CloudWatch alarm triggers on >1% failure rate. Separate alarm for 'config.validation.failure' (schema mismatch at runtime). PagerDuty integration for P1 configuration failures (kill switch, guardrail).

Rollback Event Tracking

Every rollback event recorded with: trigger (manual/automatic), triggering alarm, time-to-rollback, affected agents, root cause label. Monthly rollback report reviewed by platform team. Rollback frequency tracked as key metric.

Feature Flag Usage Analytics

Langfuse / Phoenix integration tracks which prompt versions, model IDs, and feature flags are active per agent execution. Enables correlation: configuration version X → quality metric Y. Powers configuration impact analysis for prompt and model updates.

Audit Trail

CloudTrail captures all API calls to Parameter Store, AppConfig, Secrets Manager. Custom CloudWatch Logs Insights queries for: who changed what, when, from where. Audit reports exported to S3 for compliance (SOC2, GDPR, HIPAA). 7-year retention for regulated industries.

OpenTelemetry Integration

Agent SDK instruments configuration reads with OTel spans: span attributes include config-path, config-version, cache-hit/miss, latency. Traces sent to Phoenix/Langfuse for AI-specific observability. Config read latency tracked as P99 SLO metric.

PART 13

AI-Specific Configuration Requirements

Configuration Patterns Unique to Agentic AI Systems

Agentic AI platforms require configuration capabilities that have no equivalent in traditional software systems. Prompt versioning, model routing, RAI policy management, and MCP server registries are examples of AI-specific configuration domains that must be designed from the ground up.

Prompt Registry & Versioning

- Centralized Prompt Registry stores all prompt templates with semantic versioning (MAJOR.MINOR.PATCH)
- MAJOR bump: fundamental change to prompt structure or persona
- MINOR bump: additional instructions, updated examples, clarifications
- PATCH bump: typo fixes, formatting improvements
- Prompt approval workflow: draft → review → approved → active → deprecated
- Automatic A/B testing on prompt version upgrades (10% traffic to new version, compare quality metrics)
- Rollback: instant revert to previous approved version via label switch
- Prompt templates stored in S3 (versioned), referenced by AppConfig
- Prompt rendering at runtime (late binding variables: {user_name}, {current_date})
- Prompt injection detection rules embedded in prompt registry validation

Model Registry & Routing

- Model Registry catalogs all approved models with metadata: provider, context window, cost/token, capabilities
- Model routing rules: intent profile → model selection (FAST→Haiku, BALANCED→Sonnet, ACCURATE→Opus)
- Fallback chain: primary model → fallback model → emergency model
- Model availability monitoring: circuit breaker on model errors > threshold
- Model upgrade path: canary 5% → monitor quality → expand to 50% → 100%
- Embedding model registry: separate registry for text embedding models (for RAG)
- Model cost tracking: real-time token consumption tracked per agent/model

- Model allowlist per environment: no Opus in dev (cost control), no Haiku in safety-critical workflows

MCP Server Registry

- MCP (Model Context Protocol) Server Registry: catalog of available tool servers with endpoints, capabilities, auth
- Registration includes: server ID, display name, endpoint URL, auth method, capability schema, SLA, owner
- MCP server health monitoring: synthetic probes every 60 seconds
- Agent-to-MCP connection pool management: connection limits per tenant
- MCP server version pinning: agents pin to specific server version to prevent breaking changes
- Discovery API: agents query registry for available tools matching capability requirements
- Rate limit configuration per MCP server, per tenant, per agent
- Cost tracking: external tool call costs attributed to calling agent

Knowledge Base Registry

- Catalog of all Bedrock Knowledge Bases with metadata: domain, last-indexed, doc count, embedding model
- Knowledge base access control: which agent types/projects can access which KBs
- Multi-KB routing: agent queries multiple KBs and merges results based on configured strategy
- Knowledge Base versioning: track when KB was last updated, alert on stale KBs
- Retrieval configuration per KB: top-k, similarity threshold, search type (semantic/hybrid/keyword)
- Knowledge base health: monitor embedding quality, query latency, retrieval accuracy

RAI Policy & Guardrail Configuration

- RAI (Responsible AI) Policy Registry: versioned policy documents with approval workflow
- Bedrock Guardrail ARNs registered per policy type (content, topic, PII, grounding)
- Policy hierarchy: enterprise minimum → BU policy → project policy → agent policy
- Emergency guardrail switch: instant enable of maximum restrictions across all agents
- Policy version testing: validate new RAI policies against test query set before activation
- Constitutional AI principle references: link agent configs to specific constitutional principles
- Evaluation threshold configuration: minimum RAGAS scores, human evaluation sample rates
- Safety incident playbook references: configuration links to runbooks for policy violations

Memory & Context Configuration

- Memory type registry: short-term (context window), episodic (session), semantic (vector), procedural (tool memory)
- Context budget allocation: how to split context window between system prompt, history, retrieved context, user input
- RAG retrieval configuration per agent: embedding model, top-k, similarity threshold, reranker

- Memory TTL policies: session memory expires after 24h, project memory after 30 days, user memory permanent
- Cross-agent memory sharing rules: which agents can read shared memory pools
- Memory compression configuration: when to summarize conversation history vs retain verbatim

PART 14–15

Configuration Delivery & Developer Experience

Pull/Push Models, SDKs, and Hot Reload Patterns

Configuration Delivery Models

Model	Mechanism	Latency	Complexity	Best For
Pull (Polling)	SDK polls AppConfig/SSM API every PollInterval (30–300s)	<1 second	Low	Feature flags, model config
Long Poll	AppConfig API blocks until change or timeout	Near-instant on change	Low	AppConfig native consumers
Push (EventBridge)	EventBridge rule → Lambda/SQS → Agent	<5 seconds	Medium	Kill switches, guardrails
Push (SSE)	Server-Sent Events from config API	<1 second	Medium	Real-time dashboard config
Push (WebSocket)	Bidirectional streaming connection	<100ms	High	Interactive config updates
Hybrid	Poll for normal config + push for critical flags	<100ms	Medium-High	Production enterprise (recommended)
Distributed Cache	Redis Pub/Sub + ElastiCache	<10ms	High	High-scale EKS deployments
EventBridge Pipes	EventBridge Pipes → Kinesis → agent	<10 seconds	Medium	Multi-region fan-out

Developer Experience — SDK Design

The configuration SDK is the primary interface for agent developers. It must be typed, self-documenting, testable, and support hot reload with minimal boilerplate.

```
# Python SDK – Agent Configuration Client
from agent_platform.config import ConfigClient, ConfigOptions

# Initialize – loads from AppConfig + Parameter Store + Secrets Manager
config = await ConfigClient.create(
    agent_id="customer-support-agent",
    environment="prod",
    options=ConfigOptions(
        cache_ttl_seconds=90,
        refresh_strategy="hybrid",
        poll=True,
        push_fail_open=True,
        validate_on_load=True,
        schema_validation_at_startup=True
    )
)

# Typed access – IDE autocomplete, no string key lookups
model_id = config.model.primary_model_id
temperature = config.model.temperature

# PromptRef
prompt_ref = config.prompt.system_prompt_ref
budget_usd = config.cost.monthly_budget_usd

# Feature flags
if config.flags.is_enabled("web_search_tool"):
    tools.append(WebSearchTool())

# Hot reload – callback on specific key changes
@config.on_change("model.primary_model_id")
async def on_model_change(old_val, new_val):
```

```
await reload_model_client(new_val) # Testing – mock specific values async with
config.mock({"model.temperature": 0.0, "flags.web_search_tool": False}): result = await
agent.run(test_query) assert result.model_used == config.model.primary_model_id #
Observability – all reads traced automatically (OTel) # Metrics: config.read.latency_ms,
config.cache.hit_rate, config.validation.errors
```


PART 16

Anti-Patterns

Common Enterprise Configuration Failures to Avoid

Configuration anti-patterns are the most common cause of production incidents in enterprise AI platforms. Understanding and proactively preventing these failures is as important as implementing the right patterns.

■ Configuration Sprawl
Problem: Configuration values spread across 10+ different systems with no single source of truth. Teams cannot find which service owns which configuration. Inconsistent values between services cause mysterious failures.
Solution: Solution: Implement configuration ownership model. Central registry maps every config key to its authoritative source. Configuration audit quarterly to identify orphaned values.
■ Secret Leakage via Parameter Store
Problem: API keys, passwords, and OAuth secrets stored as plain-text Parameter Store parameters. CloudTrail logs expose secret values. IAM policies too broad.
Solution: Solution: ALL secrets go to Secrets Manager. Parameter Store for non-sensitive values ONLY. Automated scanner (git-secrets, detect-secrets) runs in CI to catch accidental secret commits.
■ Hardcoded Configuration Values
Problem: Model IDs, endpoint URLs, knowledge base ARNs hardcoded in container images. Requires redeployment for any value change. Different values in different environments require different images — violates 12-Factor principles.
Solution: Solution: Zero hardcoded values. All values from external configuration. Startup validation fails fast if any required configuration is missing.
■ Environment-Specific Code Branches
Problem: if env == 'prod': use_real_model() else: use_mock(). Environment branches in application code. Configuration values embedded in conditionals. Tests don't reflect production behavior.
Solution: Solution: Externalized configuration with environment-specific values in configuration store, not in code. Feature flags for behavior differences, not environment checks.

■ Monolithic Configuration Blobs

Problem: Single JSON document containing all configuration for all agents in the platform. Every configuration change deploys the entire blob. Merge conflicts when multiple teams edit simultaneously. Blast radius of a misconfiguration is the entire platform.

Solution: Solution: Fine-grained configuration — one AppConfig profile per agent or per concern. Separate deployable units. Atomic updates with minimal blast radius.

■ Overusing AppConfig for Secrets

Problem: Storing database passwords and API keys in AppConfig freeform configurations instead of Secrets Manager. No rotation, no encryption at field level, secrets visible in AppConfig console.

Solution: Solution: AppConfig for non-sensitive configuration values only. Secrets Manager for all credentials. AppConfig stores ARN references to Secrets Manager secrets.

■ No Configuration Versioning

Problem: Configuration changes overwrite previous values with no history. When an incident is caused by a configuration change, impossible to identify what changed or roll back. No audit trail for compliance.

Solution: Solution: Immutable configuration versions. Every change creates new version. Current version labeled — rollback is just changing the label to a previous version.

■ Storing Runtime State as Configuration

Problem: Agent conversation history, session data, or dynamic counters stored in Parameter Store or AppConfig. These services are not designed for high-write state. Throttling errors in production.

Solution: Solution: Configuration services for configuration only (infrequently changing values). DynamoDB, ElastiCache, or S3 for runtime state. Clear separation of configuration and state.

■ No Configuration Testing

Problem: Configuration changes deployed directly to production without testing. Schema violations discovered at runtime. Configuration incompatibilities between services not caught until they cause agent failures.

Solution: Solution: Configuration test pipeline: schema validation → integration tests in dev → canary deployment to UAT → smoke tests → production. No configuration change skips testing.

■ Configuration Coupled with Deployment

Problem: Configuration changes require code deployments. New environment variable requires new container image. No way to update configuration without downtime. Emergency guardrail update takes 45 minutes instead of 45 seconds.

Solution: Solution: Strict separation of configuration and deployment lifecycles. AppConfig, Parameter Store, and Secrets Manager enable configuration changes completely independent of container image or Lambda function deployments.

PART 17

Best Practices Catalog

Comprehensive Configuration Governance for Enterprise Scale

Naming Standards

- Use lowercase-kebab-case for all configuration keys
- Follow hierarchy: `/layer/environment/service/category/key`
- Include version suffix for breaking changes: `/agent-config/v2/...`
- No abbreviations — prefer clarity over brevity: 'primary-model-id' not 'pm-id'
- Environment names: dev, test, uat, prod (never: d, t, u, p or development, production)
- Tag all resources: Environment, Team, Owner, CostCenter, DataClassification

Ownership & Governance

- Every configuration key has an explicit owner (team + individual)
- Configuration without an owner is considered a critical finding in quarterly audits
- Platform team owns enterprise-level and platform-level configuration
- Agent developer teams own agent-level and workflow-level configuration
- Secrets owned by security team with delegated access to consuming teams
- Configuration changes to prod require approval from both owner and security reviewer

Schema & Validation

- Every configuration type has a JSON Schema (minimum version 2020-12)
- Schema validation in CI pipeline (pre-commit hook + pipeline check)
- AppConfig Lambda validator enforces schema at deployment time
- Agent SDK validates configuration schema at startup (fail-fast)
- Schema evolution: additive changes only in MINOR versions; document breaking changes
- Type safety: use typed configuration clients (never raw dict/map access in production)

Security Baselines

- No secrets in Parameter Store — all credentials in Secrets Manager
- All configuration encrypted at rest (KMS customer-managed keys)
- Separate KMS keys per environment
- Minimum IAM permission: read-only access to specific parameter paths only

- Configuration signing: all configuration artifacts signed with asymmetric KMS key
- Audit logging mandatory: no configuration access without CloudTrail record
- Configuration changes to prod require 2-person approval (4-eyes principle)

Resilience & Caching

- Always cache configuration locally (AppConfig Agent / SDK cache)
- Implement fail-open: use last-known-good configuration on service failure
- Configuration cache TTL: 90s for feature flags, 300s for static config
- Circuit breaker on configuration service calls (open after 5 consecutive failures)
- Configuration snapshot for disaster recovery (full state backup to S3 daily)
- Test configuration service failure regularly in chaos engineering exercises

Progressive Delivery

- No configuration change goes directly to production without canary testing
- Standard rollout: dev → test → UAT → canary 5% prod → 25% → 100% prod
- Automated rollback on CloudWatch alarm (error rate > 1% or latency > SLO)
- Bake time: minimum 30 minutes at each percentage stage for prod rollouts
- Kill switch mandatory for all feature flags and capability toggles
- Configuration change freeze windows: no non-emergency changes during peak traffic

Automation & GitOps

- All configuration changes via pull request — no direct console edits in prod
- GitOps reconciliation: pipeline auto-detects and alerts on drift from Git state
- Automated testing: every PR runs configuration validation test suite
- One-click deployment to non-prod environments for fast iteration
- Automated rollback pipeline: triggered by alarm → execute rollback → notify team
- Monthly automated audits: unused configuration, expired feature flags, orphaned secrets

PART 18

Comprehensive Comparison Matrix

40+ Decision Criteria Across 14 Configuration Solutions

The following matrix evaluates 14 configuration management solutions across 40+ criteria relevant to Enterprise Agentic AI platforms on AWS. Scores are 1–5 (5 = best). Use this matrix to select the right tool for each configuration category.

	SSM Param Store	AWS A ppConf ig	Secrets Manag er	Dynam oDB Config Svc	HashiC orp Vault	Launch Darkly	Open Feature	K8s Co nfigMa ps	Spring Cl. Config	Consul
PERFORMANCE										
Read Latency	4	5	3	5	3	5	4	5	3	4
Write Latency	4	3	4	4	3	4	4	4	3	4
Throughput	3	4	4	5	4	5	4	5	3	4
Cache Support	3	5	3	5	3	5	3	4	4	5

	SSM Param Store	AWS A ppConf ig	Secrets Manag er	Dynam oDB Config Svc	HashiC orp Vault	Launch Darkly	Open Feature	K8s Co nfigMa ps	Spring Cl. Config	Consul
FEATURES										
Hot Reload	2	5	3	5	4	5	4	2	4	5
Feature Flags	1	4	1	3	2	5	5	1	2	3
A/B Testing	1	3	1	3	2	5	3	1	1	2
Kill Switches	2	5	1	4	3	5	5	2	3	4
Canary Rollout	2	5	1	4	2	5	4	2	3	3
Versioning	4	4	4	4	4	4	3	2	4	4
Rollback	3	5	4	4	4	5	4	2	4	4
Hierarchy	4	3	2	5	4	3	3	2	5	4
Schema Validation	2	4	2	5	3	4	4	2	4	3

	SSM Param Store	AWS A ppConf ig	Secrets Manag er	Dynam oDB Config Svc	HashiC orp Vault	Launch Darkly	Open Feature	K8s Co nfigMa ps	Spring Cl. Config	Consul
SECURITY										

Encryption at Rest	4	4	5	4	5	4	3	3	3	4
Secret Management	2	1	5	2	5	1	1	1	2	3
IAM Integration	5	5	5	4	3	4	3	4	3	3
Audit Logging	4	4	5	4	5	4	3	2	3	4
RBAC/ABAC	4	4	5	4	5	4	3	4	3	4
Zero Trust	3	3	4	4	5	4	3	3	3	4

OPERATIONS	SSM Param Store	AWS A ppConf ig	Secrets Manag er	Dynam oDB Config Svc	HashiC orp Vault	Launch Darkly	Open Feature	K8s Co nfigMa ps	Spring Cl. Config	Consul
Ease of Use	4	4	4	2	2	5	4	4	3	3
Operational Overhead	5	4	4	2	1	5	4	4	3	3
Multi-Region	3	3	4	5	4	4	3	3	3	4
Multi-Account	4	4	4	4	4	4	3	3	2	3
DR / HA	4	4	5	5	4	5	4	3	3	4
Disaster Recovery	3	4	4	4	4	5	3	2	3	4

GOVERNANCE	SSM Param Store	AWS A ppConf ig	Secrets Manag er	Dynam oDB Config Svc	HashiC orp Vault	Launch Darkly	Open Feature	K8s Co nfigMa ps	Spring Cl. Config	Consul
Change Management	3	5	4	4	4	5	4	2	4	4
Approval Workflows	2	4	3	4	3	5	3	1	3	3
Configuration Drift	3	4	3	4	4	3	3	3	3	3
Compliance Reports	4	4	5	4	4	4	3	2	3	3

ECOSYSTEM	SSM Param Store	AWS A ppConf ig	Secrets Manag er	Dynam oDB Config Svc	HashiC orp Vault	Launch Darkly	Open Feature	K8s Co nfigMa ps	Spring Cl. Config	Consul
AWS Native	5	5	5	4	3	3	3	3	2	3
SDK Coverage	5	4	5	3	4	5	5	5	5	5
Terraform Support	5	5	5	4	5	4	3	5	4	4
CloudFormation	5	5	5	3	3	3	2	3	3	3
Cost (1=expensive)	5	4	4	3	2	2	5	5	4	4

Legend: 5=Excellent 4=Good 3=Adequate 2=Limited 1=Poor/Not Supported

PART 19

Production Reference Architecture

Enterprise Agent Builder Platform on AWS

The following reference architecture defines a production-grade configuration management system for an Enterprise Agent Builder Platform on AWS. It is designed for multi-account AWS Organizations, multi-region deployment, and supports thousands of concurrent AI agents across multiple teams and tenants.

Architecture Overview

LAYER 1: Configuration Control Plane (Central Tooling Account)
→ Git Repository (CodeCommit / GitHub): Single source of truth for all configuration
→ AWS CodePipeline: CI/CD for configuration changes with validation gates
→ AWS CodeBuild: Linting, schema validation, policy checks, integration tests
→ Parameter Store (us-east-1 primary): Enterprise and platform-level configuration
→ AppConfig Application 'AgentPlatform': All feature flags and dynamic configuration
→ Secrets Manager: Centralized secrets with cross-account resource policies
→ S3 Configuration Archive: Configuration snapshots, schemas, prompt templates
→ AWS Config: Compliance rules and drift detection
→ CloudTrail (organization-level): Immutable audit log for all configuration API calls
→ EventBridge (default event bus): Configuration change event routing
LAYER 2: Configuration Distribution Layer
→ EventBridge Rules: Route configuration.* events to downstream consumers
→ SNS Topic 'config-changes': Fan-out to per-environment SQS queues
→ SQS Queues per environment: Guaranteed delivery of configuration change notifications
→ Lambda 'config-propagation': Processes change events, invalidates caches, notifies agents
→ ElastiCache (Redis): Distributed configuration cache for EKS agent fleets
→ CloudFront + S3: Global distribution of large configuration artifacts (schemas, prompts)

LAYER 3: Workload Accounts (Dev / Test / UAT / Prod)

- Parameter Store (local): Environment-specific overrides, inherited from central
- AppConfig Agent Extension: Deployed in Lambda layers and ECS sidecars
- EKS Cluster: Agent pods with AppConfig sidecar container and Redis cache client
- ECS Services: Agent containers with AppConfig Extension Lambda layer
- Lambda Functions: Agent handlers with AppConfig SDK + SSM Parameter caching
- Bedrock AgentCore Runtime: Managed agents with built-in config integration
- Bedrock Knowledge Bases: Referenced via configuration (ARN stored in Parameter Store)
- Bedrock Guardrails: ARNs managed in configuration, applied per-agent-type

LAYER 4: Observability & Governance

- CloudWatch Metrics: Configuration read latency, cache hit rates, propagation delay
- CloudWatch Alarms: Trigger auto-rollback on error rate spike post-deployment
- OpenTelemetry Collector (EKS DaemonSet): Traces configuration reads as spans
- Phoenix / Langfuse: AI-specific observability with configuration version correlation
- AWS Security Hub: Configuration compliance findings aggregated centrally
- Grafana Dashboard: Configuration health, propagation status, feature flag usage

Configuration Publishing Sequence

Step	Actor	Action
1	Developer	git push to feature branch with configuration change in YAML
2	CodePipeline	Triggers on PR create: run schema validation, naming check, security scan
3	CodeBuild	JSON Schema validation passes. OPA policy check passes. Naming convention valid.
4	Human Reviewer	Platform engineer reviews PR. Approves for non-prod environments.
5	CodePipeline	Merge to main triggers deployment pipeline
6	CodeBuild	Apply configuration to dev: Parameter Store put, AppConfig create-deployment
7	AppConfig	Deployment starts with LINEAR_50_PERCENT_EVERY_30_SECONDS strategy in dev
8	AppConfig Agent	Running Lambda/ECS instances receive updated configuration via long-poll
9	Integration Tests	Automated tests validate configuration is applied correctly in dev
10	Pipeline Gate	Manual approval required to promote to prod
11	Production Approval	BU CTO or security reviewer approves production deployment

Step	Actor	Action
12	CodePipeline	Deploys to prod with CANARY_10_PERCENT_20_MINUTES strategy
13	CloudWatch Alarm	Monitors error_rate and latency. If alarm triggers → auto-rollback
14	EventBridge	Publishes configuration.published event to all subscribers
15	Propagation Lambda	Invalidates Redis cache, notifies EKS agents via SQS
16	Audit Log	CloudTrail records complete deployment with approver identity

Multi-Account AWS Organizations Architecture

- **Management Account:** AWS Organizations, SCPs that prevent direct configuration API calls bypassing pipeline, centralized CloudTrail, Security Hub aggregation
- **Tooling Account:** CodePipeline, CodeBuild, central Parameter Store, central Secrets Manager, AppConfig application definitions, S3 configuration archive
- **Shared Services Account:** ElastiCache (Redis) for shared configuration cache, EventBridge custom event bus, SNS/SQS for configuration propagation
- **Dev Account:** Dev environment Parameter Store overrides, AppConfig dev environment, all workloads for development
- **Test Account:** Test environment configuration, synthetic credentials, integration test infrastructure
- **UAT Account:** UAT environment configuration, production-like settings with test data, acceptance testing
- **Production Account:** Production configuration, real credentials (cross-account from Tooling Account), production workloads
- **Log Archive Account:** All CloudTrail logs consolidated for 7-year immutable retention, compliance reports

High Availability & Disaster Recovery

- Parameter Store: Cross-region SSM parameter replication via EventBridge + Lambda automation
- AppConfig: Deployed independently per region — each region has full AppConfig stack
- Secrets Manager: Multi-region secret replication enabled for all production secrets
- ElastiCache: Redis Global Datastore for <1s cross-region configuration cache replication
- DynamoDB (custom config service): Global Tables for active-active multi-region configuration
- S3 Configuration Archive: Cross-region replication with S3 versioning and MFA delete
- RTO Target: <15 minutes for full configuration plane restoration in DR region
- RPO Target: <5 minutes for configuration data (DynamoDB Global Tables + S3 replication)
- Configuration snapshot: Full configuration state backed up to S3 every 6 hours
- Runbook: Automated DR failover playbook tested quarterly via chaos engineering exercises

Appendix A: Decision Matrix — Technology Selection

This decision matrix provides the authoritative mapping of configuration categories to recommended AWS services and patterns for the Enterprise Agent Builder Platform.

Configuration Category	Primary Service	Secondary	Rationale
LLM Model Selection	AWS AppConfig	DynamoDB Config Svc	Hot reload required; canary rollout for model upgrades
Foundation Model IDs	Parameter Store (Advanced)	AppConfig	Infrequent change; hierarchical path per env/agent
Prompt Templates	S3 (versioned) + Prompt Registry	AppConfig	Large documents; versioning critical; Git-managed
Prompt Version Pinning	AppConfig Feature Flags	Parameter Store	Hot swap; progressive rollout; instant rollback
Knowledge Base ARNs	Parameter Store	AppConfig	ARNs are stable; hierarchical by project/agent
MCP Server Endpoints	Parameter Store + Service Registry	DynamoDB	Discovery API required; health monitoring needed
Tool Endpoints	Parameter Store	AppConfig	Stable URLs; environment-specific overrides
API Keys (3rd party)	Secrets Manager	—	Rotation mandatory; audit required; never in SSM
OAuth Client Secrets	Secrets Manager	Vault	Rotation; short-lived token support
Database Passwords	Secrets Manager (auto-rotate)	—	RDS native rotation; mandatory encryption
Feature Flags (agents)	AWS AppConfig + Evidently	LaunchDarkly	Progressive rollout; per-tenant targeting
Kill Switches	AppConfig + EventBridge	PushLaunchDarkly	<5s propagation; no targeting rules needed
RAI Policy References	Parameter Store	AppConfig	Policy ARNs stable; version labels for rollback
Guardrail Config ARNs	Parameter Store	AppConfig	Emergency update path via AppConfig if needed
Token / Cost Limits	AppConfig	DynamoDB	Hot update when budget events trigger; per-tenant
Retry / Timeout Policies	Parameter Store	AppConfig	Stable; environment-specific; infrequent change
Regional Endpoints	Parameter Store (by region)	Cloud Map	Region-specific paths; Service Discovery for dynamic
OTel Collector Endpoints	Parameter Store	—	Infrastructure configuration; env-specific
Langfuse / Phoenix Config	Parameter Store + Secrets Manager	—	Endpoint in SSM; API key in Secrets Manager
Vector DB Configuration	Parameter Store + Secrets Manager	—	Endpoint/index in SSM; credentials in SM
Memory Configuration	AppConfig	Parameter Store	May change per experiment; hot reload useful
Human Approval Thresholds	AppConfig	DynamoDB	Emergency lowering required; hot reload critical
Multi-Agent Routing Rules	AppConfig + DynamoDB	—	Complex rules; searchable; version history
A/B Test Configuration	CloudWatch Evidently	LaunchDarkly	Native statistical analysis; traffic splitting
Tenant Configuration	DynamoDB Config Svc	Parameter Store	Per-tenant complexity; query patterns needed
Environment-Specific Values	Parameter Store (env-scoped)	AppConfig	Hierarchy at env level; stable values
Schema Definitions	S3 (versioned, public)	CodeArtifact	Large files; versioned; shared read access

Configuration Category	Primary Service	Secondary	Rationale
Certificate / TLS Config	ACM + Parameter Store (ARNs)	Amazon PKI	ACM manages cert lifecycle; ARNs in SSM
Encryption Key References	Parameter Store (KMS ARNs)		KMS key ARNs; never secrets themselves
Infrastructure ARNs	CloudFormation Outputs + SSM	CloudFormation Remote State	Cloud outputs → SSM for cross-stack reference

Appendix B: Implementation Roadmap

Phase 1: Proof of Concept		0–90 Days
Goal	Validate key technology choices; demonstrate hot reload for one agent type	
Parameter Store	Implement naming hierarchy for one service. Migrate 10 hardcoded values to SSM.	
AppConfig	Deploy one feature flag (kill switch for new agent). Test hot reload <90s.	
Secrets Manager	Migrate 3 API keys from Parameter Store/env vars to Secrets Manager.	
SDK PoC	Build typed Python configuration client for one agent. Demonstrate mock support for testing.	
GitOps	Set up CodePipeline for one configuration path. First PR-based configuration deployment.	
Observability	CloudWatch dashboard: config reads, cache hits, propagation time.	
Success Metrics	Hot reload working (<90s). Zero hardcoded secrets. First GitOps deployment complete.	
Phase 2: Minimum Viable Platform		90–180 Days
Goal	Configuration Control Plane supporting all existing agents; team self-service operational	
Hierarchy	Implement 8-level hierarchy (Enterprise → Agent). All agents migrated.	
AppConfig	All feature flags migrated to AppConfig. EventBridge push for kill switches.	
SDK GA	Python and TypeScript SDK v1.0 released. Documented. Internal teams onboarded.	
CI Validation	Schema validation in all CI pipelines. Naming convention enforced.	
Self-Service Portal	Backstage plugin for agent configuration. Form-based creation.	
Prompt Registry	v1 Prompt Registry operational. 10 prompts migrated. Versioning working.	
Security Baseline	KMS encryption for all config. IAM ABAC policies. Audit logging.	
Success Metrics	100% agents using config platform. 0 hardcoded values. P99 read <5ms.	
Phase 3: Enterprise Scale		180–365 Days
Goal	Full governance, progressive delivery, and multi-team self-service at enterprise scale	
Full Hierarchy	All 14 hierarchy levels implemented. Inheritance engine with conflict resolution.	
Progressive Delivery	Canary rollout for all configuration types. Auto-rollback on alarms.	

DynamoDB Config Svc	Deploy for complex schemas (multi-tenant, routing rules). DAX caching.
LaunchDarkly	Advanced feature flags with per-tenant targeting and A/B testing.
Model Registry	Complete Model Registry with routing, fallback, and cost tracking.
MCP Registry	MCP Server Registry with discovery API and health monitoring.
Compliance	SOC2 audit trail. Configuration lineage reports. Automated compliance checks.
Developer Platform	CLI v2, Terraform provider, Configuration Marketplace launched.
Success Metrics	1000+ agents managed. <30s propagation P99. 99.99% config availability.

Phase 4: Global Multi-Region Platform		12–24 Months
Goal	Global deployment with sovereign support, AI-optimized configuration, and marketplace	
Multi-Region	Active-active configuration in 3+ regions. DynamoDB Global Tables. Redis Global Datastore.	
Sovereign Deployments	On-premise configuration replica for air-gapped deployments. No external dependencies.	
Config Marketplace	Cross-team reusable configuration components. Rating and discovery.	
AI-Optimized Config	ML-driven automatic configuration optimization based on performance metrics.	
Intent Profiles	FAST/SAFE/CHEAP/BALANCED profiles with automated model and parameter selection.	
Configuration Analytics	AI-powered anomaly detection in configuration changes. Predictive rollback.	
Zero-Downtime Schema Migration	Automated dual-write migration for breaking schema changes.	
Success Metrics	10,000+ agents. 5 regions. 99.999% configuration availability. <1s global propagation.	

Appendix C: RFC Template & Implementation Checklist

Configuration Change RFC Template

```
RFC-{NUMBER}: {TITLE} ██████████ METADATA
Status: DRAFT | REVIEW | APPROVED | IMPLEMENTED | REJECTED Author: {name} <{email}>
Reviewers: {platform-engineer}, {security-reviewer}, {business-owner} Created:
{YYYY-MM-DD} Target Deploy: {YYYY-MM-DD} (prod) Priority: P0 (emergency) | P1 (this
sprint) | P2 (this quarter) SUMMARY {One paragraph describing the configuration change
and its purpose.} MOTIVATION Problem being solved: Current state (how is this handled
today): Consequences of not making this change: CONFIGURATION CHANGES PROPOSED Service: [
] Parameter Store [ ] AppConfig [ ] Secrets Manager [ ] DynamoDB Config Svc [ ] Other:
_____ Configuration Path(s): - /env/{env}/platform/{service}/{key} [BEFORE: value]→
[AFTER: value] Schema Changes: [ ] None [ ] Additive (MINOR bump) [ ] Breaking (MAJOR
bump) Hierarchy Level: [ ] Enterprise [ ] Org [ ] BU [ ] Platform [ ] Environment [ ]
Region [ ] Tenant [ ] Project [ ] Agent [ ] Workflow ROLLOUT STRATEGY Strategy: [ ]
AllAtOnce [ ] Linear_10%/10min [ ] Canary_5%/30min [ ] Custom Rollback Plan: {Describe
exact rollback steps and time-to-rollback estimate} Rollback Time: {estimated time to
complete rollback} Kill Switch: [ ] Yes (flag: {flag-name}) [ ] No (explain why not
needed) RISK ASSESSMENT Blast Radius: [ ] Single agent [ ] All agents of type X [ ] All
tenants [ ] Global Risk Level: [ ] LOW [ ] MEDIUM [ ] HIGH [ ] CRITICAL Breaking Change:
[ ] Yes (requires consumer migration) [ ] No Data Migration: [ ] Yes (describe migration
steps) [ ] No Downtime Risk: [ ] None [ ] Brief (<1 min) [ ] Significant (planned
maintenance) VALIDATION Schema Validated: [ ] Yes (CI pipeline) Naming Convention: [ ]
Yes (reviewed by platform team) Security Review: [ ] Yes (security team sign-off) Test
Environment: [ ] Yes (tested in dev/test since {date}) Integration Tests: [ ] Pass [ ]
Fail (describe) OBSERVABILITY Monitoring Alert: [ ] CloudWatch alarm created for this
change Propagation Check: [ ] Automated propagation test configured Rollback Alarm: [ ]
Auto-rollback on {metric} > {threshold} Audit Trail: [ ] CloudTrail logging verified
APPROVALS Platform Engineer: _____ Date: _____ Security Reviewer:
_____ Date: _____ Business Owner: _____ (for P0/P1 prod changes)
Deployment by: _____ Date: _____
```

Implementation Checklist — New Agent Configuration

Design Phase

- [] Agent schema defined using platform JSON Schema template
- [] Configuration hierarchy level determined (which levels need overrides)
- [] Naming convention followed for all parameter paths
- [] Secret vs non-secret classification done for all values
- [] Owner assigned to all configuration keys
- [] Schema reviewed by platform team

Development Phase

- [] Configuration defined in YAML file in GitOps repository
- [] Schema validation passes in local dev
- [] SDK integrated in agent code (no raw API calls in application code)
- [] Hot reload handler implemented for critical configuration keys

- ☐ Mock configuration implemented for unit tests
- ☐ Integration test verifies configuration is read correctly
- ☐ No hardcoded values in container image or Lambda package

Security Review

- ☐ All secrets in Secrets Manager (none in Parameter Store or environment variables)
- ☐ IAM role scoped to minimum required configuration paths
- ☐ KMS encryption verified for all configuration stores
- ☐ Configuration signing enabled if classified as SENSITIVE
- ☐ Cross-account access policy reviewed if applicable
- ☐ Audit logging verified in CloudTrail

Pre-Production Deployment

- ☐ Configuration deployed to dev via CI pipeline (not manually)
- ☐ Integration tests pass in dev environment
- ☐ Schema validation passes in AppConfig Lambda validator
- ☐ Configuration promoted to test environment
- ☐ UAT testing completed with production-equivalent configuration
- ☐ RFC created and approved for production deployment

Production Deployment

- ☐ Production deployment uses AppConfig deployment strategy (canary or linear)
- ☐ CloudWatch alarms configured for auto-rollback
- ☐ On-call engineer notified of deployment start
- ☐ Propagation monitoring dashboard reviewed post-deployment
- ☐ All running agent instances confirmed to have received new configuration
- ☐ Deployment status documented in incident tracking system

Post-Deployment

- ☐ Configuration visible in Internal Developer Platform portal
- ☐ Documentation updated in team runbook
- ☐ Ownership recorded in configuration registry
- ☐ Monitoring alert thresholds calibrated based on baseline metrics
- ☐ Feature flag expiry date set if applicable
- ☐ Next review date scheduled for configuration